HDOC Day-64 Activity

Finding the Repeated Element Using XOR and Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-64

Aim

To write a C program that finds the only repeated element in a variable-length integer array using the XOR technique. The array is passed to **duplicate_element()** using its address and accessed through pointers.

Problem Requirements

- Read the size and elements of a variable-length integer array **nums[]**.
- Pass the array to **duplicate_element()** using its address.
- Find and display the only repeated element.
- Use an efficient XOR-based approach and process the array in one traversal.
- Assume every other element appears exactly once.

Input Pattern Used

From the supplied examples, an array of size n contains the values **1 through n-1**, with exactly one of those values appearing twice. This known range allows XOR to isolate the repeated value.

Why XOR Works

XOR has useful cancellation properties: **x XOR x = 0** and **x XOR 0 = x**. XORing every array element and then XORing all expected values from 1 through n-1 cancels every value that occurs once in both groups. The duplicated value is the only value left.

Algorithm

1. Read array size n and the n elements of nums[].
2. Pass nums and n to **duplicate_element(nums, n)**.
3. Initialize result = 0.
4. Traverse the array once and XOR every element into result using ***(nums+i)**.
5. XOR result with every expected value from 1 through n-1.
6. All non-repeated values cancel, leaving only the duplicate.
7. Display the repeated element.

C Program

```
#include <stdio.h>

void duplicate_element(int *nums, int n)
{
    int result = 0;

    for (int i = 0; i < n; i++)
    {
        result = result ^ *(nums + i);
    }

    for (int i = 1; i <= n - 1; i++)
    {
        result = result ^ i;
    }

    printf("Repeated element: %d", result);
}

int main()
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    duplicate_element(nums, n);

    return 0;
}
```

Pointer Usage

duplicate_element(nums, n) passes the address of the first element. The parameter **int *nums** receives this address, and ***(nums+i)** accesses nums[i] using pointer arithmetic.

Example 1 XOR Dry Run

Step	Operation	Meaning
Array XOR	1 XOR 3 XOR 4 XOR 2 XOR 2	XOR all given values
Expected XOR	1 XOR 2 XOR 3 XOR 4	XOR expected range 1 to 4
Combined	(1^3^4^2^2) ^ (1^2^3^4)	Equal values cancel
Result	2	Repeated element

For **1 3 4 2 2**, all values from 1 to 4 cancel with their expected copies except the extra 2. Therefore, the repeated element is **2**.

Test Cases / Expected Results

Case	Input Array	Expected Output
Example 1	1 3 4 2 2	Repeated element: 2
Example 2	3 1 3 4 2	Repeated element: 3
Example 3	1 4 5 2 3 4	Repeated element: 4

Example 2 Explanation

The size is 5, so the expected values are 1, 2, 3, and 4. In **3 1 3 4 2**, the value 3 appears twice. XOR cancellation leaves **3**.

Example 3 Explanation

The size is 6, so the expected values are 1 through 5. In **1 4 5 2 3 4**, all expected values cancel and the extra **4** remains.

Complexity

Time Complexity: O(n). The input array is traversed once, and the known range 1 to n-1 is also processed linearly.

Space Complexity: O(1) extra space because only the XOR result and loop variables are used.

Result

The program successfully finds the single repeated element using pointer-based array access and XOR cancellation. It preserves constant extra space and linear running time.

Execution Example

```
Enter the size of array: 5
Enter the elements: 1 3 4 2 2
Repeated element: 2
```